



CYBERSTER

Cyber Security Internship Program

INTERNSHIP WEEKLY REPORT

Week 1 — Reconnaissance & OSINT Fundamentals (Red Team Track)

Topic: Passive & Active Subdomain Enumeration, DNS/WHOIS Enumeration, Technology Fingerprinting, Search-Engine & Code-Repository OSINT, and Visual Reconnaissance — performed against HackThisSite.org, a publicly authorized legal hacking-practice platform

Student Name: Umar Farooq

Batch: B#2

Instructor: Umar Niaz

Submission Date: June 25, 2026

Contents

Executive Summary	3
Learning Objectives	4
Environment Setup.....	4
Operating System	4
Virtualization Software & VM Configuration	4
Network Type	4
Lab / Engagement Architecture	4
Installed Tools.....	5
Task Documentation	6
Task 1: Passive Subdomain Enumeration (OSINT Sources).....	6
Task 2: Data Consolidation, Deduplication & Master Target List	9
Task 3: DNS Resolution, IP Mapping & DNS Record Enumeration	11
Task 4: WHOIS & Domain Registration Enumeration.....	14
Task 5: Technology Fingerprinting.....	15
Task 6: Search-Engine Reconnaissance (Google Dorking)	18
Task 7: GitHub Reconnaissance (Code & Secret Search)	21
Task 8: HTTP Probing & Live Host Discovery	24
Task 9: Visual Reconnaissance with GoWitness	26
Technical Concepts Learned	33
Challenges Faced	35
Best Practices	36
Weekly Summary Table	37
Conclusion	38
References.....	39
Appendix.....	40
A. Raw Evidence Files Retained.....	40
B. Full Screenshot Archive	40
C. Terminal & Browser Screenshot Index	40
D. Submission Checklist	40

Executive Summary

During the first week of my Cyber Security Internship at Cyberster, on the Red Team track, I was introduced to the reconnaissance phase of the penetration testing methodology, with a specific focus on Open Source Intelligence (OSINT) gathering and subdomain enumeration. Rather than starting with theory alone, the week was built around a hands-on engagement against HackThisSite.org, a long-running, publicly available, and explicitly authorized legal hacking-practice platform, which gave me a realistic target without any of the legal or ethical ambiguity of testing infrastructure I do not have permission to touch.

Working inside a Kali Linux environment, I progressively built a reconnaissance pipeline over nine practical tasks: passive subdomain discovery with Subfinder, Assetfinder, Amass, and crt.sh certificate-transparency search; consolidation and deduplication of all sources into a single master target list; DNS resolution and IP mapping; WHOIS and DNS-record enumeration; technology fingerprinting with WhatWeb and Wappalyzer; search-engine and GitHub-based OSINT through Google dorking and GitHub code search; live-host discovery using httpx and httpprobe; and finally large-scale visual reconnaissance with GoWitness, which screenshots every reachable web asset and indexed the results into a local reporting database.

By the end of the week I had compiled a master list of 72 valid candidate subdomains, resolved 52 unique IP addresses, confirmed 54 live HTTP/HTTPS endpoints, and generated 62 screenshots together with 681 captured headers and 905 network-log entries inside the GoWitness database. Just as importantly, I learned how individual reconnaissance tools complement each other, why raw OSINT output has to be cleaned and cross-checked before it becomes usable intelligence, and how a negative result (no leaked secret found, no open directory listing) is still a real, reportable finding. I close Week 1 with a working, repeatable recon toolkit and a much clearer picture of how the opening stage of a red-team engagement is actually carried out in practice.

Learning Objectives

By the end of Week 1, I was expected to be able to:

- Explain the role of reconnaissance within the wider penetration testing lifecycle and relate it back to the CIA Triad it ultimately protects.
- Work comfortably inside a Kali Linux terminal environment to chain multiple security tools together.
- Apply passive OSINT techniques that gather information without directly touching target infrastructure (Amass passive mode, crt.sh, GitHub code search).
- Apply active reconnaissance techniques that do interact with target infrastructure (DNS resolution, WHOIS, HTTP probing).
- Perform subdomain enumeration using multiple independent tools and understand why their results must be merged and deduplicated rather than trusted individually.
- Fingerprint the web technologies and server stack of a target and judge their security relevance.
- Use search engines and code-hosting platforms (Google, GitHub) as structured reconnaissance sources, including triaging false positives.
- Produce a documented, repeatable reconnaissance workflow and evidence trail that could be handed to a supervisor or client as-is.

Environment Setup

Operating System

Kali Linux (rolling release), accessed directly from a terminal multiplexer (Kitty/terminal session shown as mutex@kali in every screenshot). All recon work for the week was carried out from the working directory ~/Desktop/Cyberster/Red Team/week1.

Virtualization Software & VM Configuration

This week's task set did not include any fresh VM provisioning, ISO installation, or hypervisor configuration screenshots — the work was carried out inside an already-provisioned Kali Linux install carried over from earlier onboarding. I am noting this explicitly rather than inventing installation evidence that does not exist for Week 1.

Network Type

Standard internet-connected host networking, with outbound DNS resolution explicitly pinned to four public resolvers (1.1.1.1, 8.8.8.8, 9.9.9.9, and 208.67.222.222) for tools such as Amass that support custom resolver lists, ensuring consistent and uncached answers across the different enumeration tools.

Lab / Engagement Architecture

The architecture for the week was simple by design: an attacker workstation (Kali Linux) on the open internet, querying the public DNS and certificate-transparency ecosystem, and finally interacting with the

target organization's edge infrastructure — a mix of Fastly, GitHub Pages, and Varnish-fronted services plus an origin IP block in the 137.74.187.0/24 range.

Installed Tools

The table below lists every tool used during Week 1, in the order it first appears in the task documentation.

Tool	Version / Type	Purpose
Subfinder	v2.14.0	Passive subdomain enumeration via aggregated OSINT sources
Assetfinder	CLI, latest	Passive subdomain discovery via certificate & API sources
OWASP Amass	Passive & active engine	Subdomain enumeration and DNS/ASN relationship mapping
crt.sh + curl + jq	Web service / CLI	Certificate Transparency log search
dig / nslookup	dnsutils (bind9)	DNS record resolution (A, MX, NS, TXT)
whois	CLI	Domain registration and registrar lookup
WhatWeb	CLI, aggression level 3	Server-side web technology fingerprinting
Wappalyzer	Browser extension	Client-side technology fingerprinting (cross-check)
httpx	v1.9.0	HTTP probing with status code, title, and tech-detect
httprobe	CLI	Lightweight live/dead host probing
GoWitness	v3	Mass website screenshotting, header/network logging, dashboard

Task Documentation

Each task below follows the same structure: objective, theory, tools, step-by-step procedure, exact commands used, output/results, supporting screenshots, observations, key learning points, and security relevance. All nine tasks were performed against hackthissite.org, whose own homepage explicitly describes itself as “a free, safe and legal training ground for hackers to test and expand their ethical hacking skills,” which is why it was used as this week's live target instead of a purely simulated lab.

Task 1: Passive Subdomain Enumeration (OSINT Sources)

Objective

Identify as many subdomains of hackthissite.org as possible without sending direct probing traffic to the target's own infrastructure, relying only on publicly indexed third-party sources.

Theory / Background

Passive reconnaissance collects information about a target exclusively from third parties — search engines, certificate transparency logs, DNS aggregators, and public APIs — so the target itself never sees the activity in its own server logs. This is normally the very first step of the reconnaissance phase in a standard penetration testing methodology, performed before any active technique that the target could detect.

Tools Used

Subfinder v2.14.0, Assetfinder, OWASP Amass (passive mode), crt.sh via curl + jq.

Procedure

1. Ran Subfinder against the root domain with the `-all` flag (queries every configured source) and saved the output to `subfinder.txt`.
2. Ran Assetfinder with `--subs-only` to restrict results to subdomains, piping through `tee` so the output was both saved and visible live.
3. Ran Amass in passive enumeration mode against the domain, which queries cached/third-party sources and prints discovered DNS relationships (CNAME, NS, MX, A records) rather than a flat list.
4. Queried crt.sh's JSON API directly with curl, extracted the `name_value` field with jq, deduplicated with `sort -u`, and saved the result to `crtsh.txt`.

Commands Used

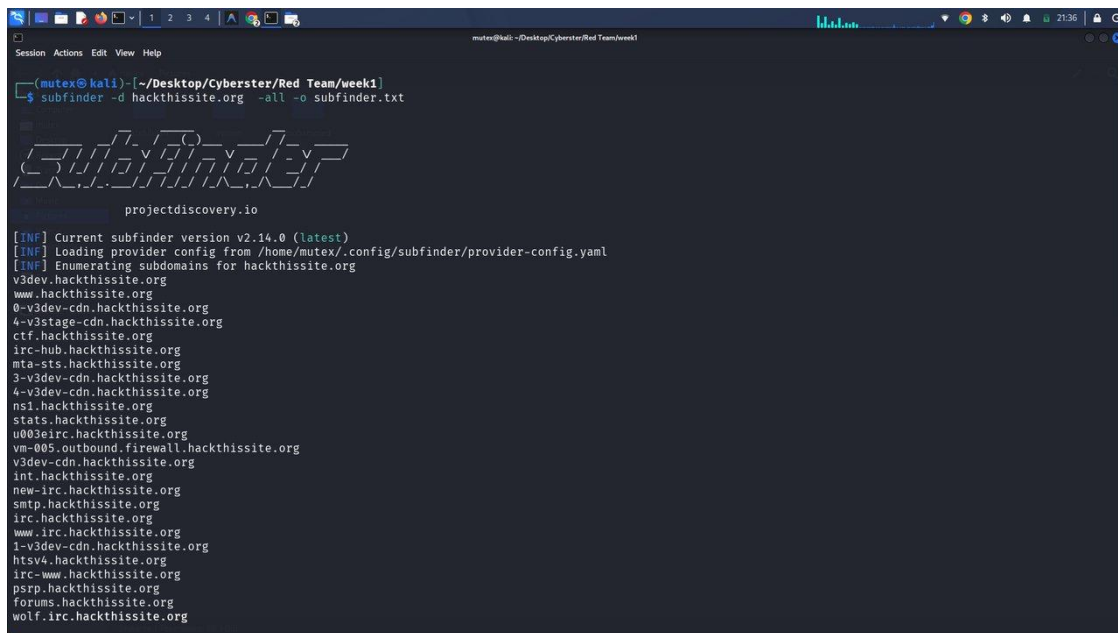
```
subfinder -d hackthissite.org -all -o subfinder.txt
assetfinder --subs-only hackthissite.org | tee assetfinder.txt
amass enum -passive -d hackthissite.org
curl -s "https://crt.sh?q=%hackthissite.org&output=json" | jq -r '.[].name_value' | sort -u > crtsh.txt
```

Output / Results

Subfinder returned 69 candidate names, Assetfinder returned 29, and crt.sh returned 21 certificate-derived names (including a wildcard `*.hackthissite.org` entry). Amass's passive run additionally mapped DNS

relationships rather than a flat list, surfacing details such as the site's Google-Workspace mail routing and its BuddyNS-hosted name servers.

Screenshots & Evidence: As shown in Figure 1, Subfinder printed its enumeration banner and streamed subdomains in real time. Figure 2 captures the Assetfinder run, Figure 3 shows Amass passive mode resolving DNS relationships, and Figure 4 documents the crt.sh certificate-transparency query and resulting subdomain list.

A terminal window on a Kali Linux system showing the execution of the subfinder tool. The command entered is 'subfinder -d hackthissite.org -all -o subfinder.txt'. The output displays the subfinder banner, the version (v2.14.0), and a list of discovered subdomains for hackthissite.org. The subdomains listed include: v3dev.hackthissite.org, www.hackthissite.org, 0-v3dev-cdn.hackthissite.org, 4-v3stage-cdn.hackthissite.org, ctf.hackthissite.org, irc-hub.hackthissite.org, mta-sts.hackthissite.org, 3-v3dev-cdn.hackthissite.org, 4-v3dev-cdn.hackthissite.org, ns1.hackthissite.org, stats.hackthissite.org, u003eirc.hackthissite.org, vm-005.outbound.firewall.hackthissite.org, v3dev-cdn.hackthissite.org, int.hackthissite.org, new-irc.hackthissite.org, smtp.hackthissite.org, irc.hackthissite.org, www-irc.hackthissite.org, 1-v3dev-cdn.hackthissite.org, htsw4.hackthissite.org, irc-www.hackthissite.org, psrp.hackthissite.org, forums.hackthissite.org, and wolf.irc.hackthissite.org.

```
mutex@kali:~/Desktop/Cyberster/Red Team/week1
$ subfinder -d hackthissite.org -all -o subfinder.txt

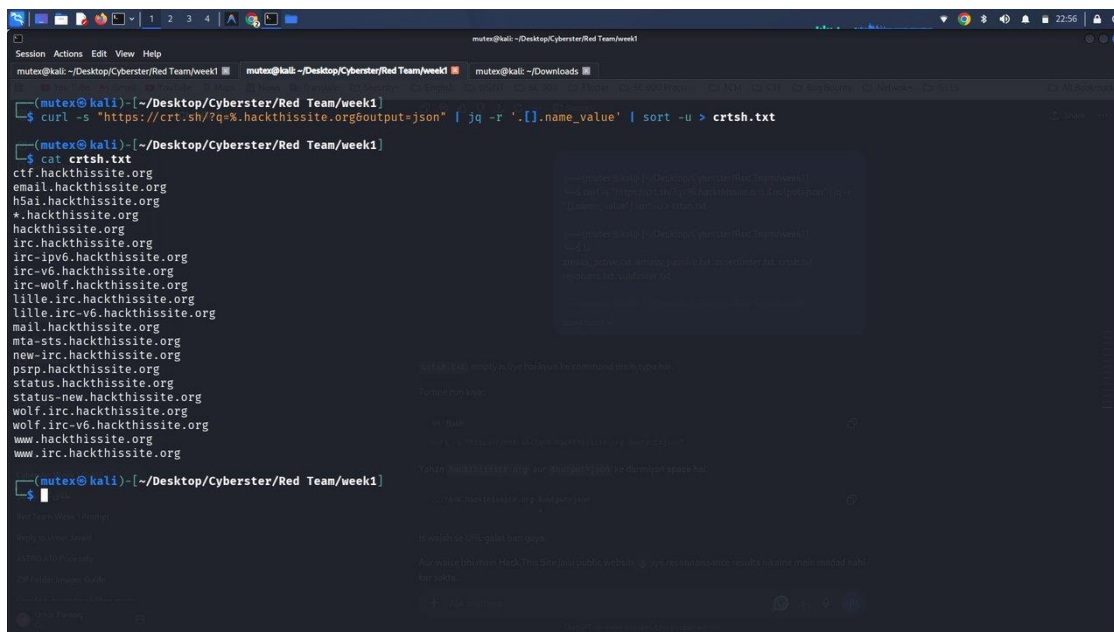
  Subfinder
  projectdiscovery.io

[INF] Current subfinder version v2.14.0 (latest)
[INF] Loading provider config from /home/mutex/.config/subfinder/provider-config.yaml
[INF] Enumerating subdomains for hackthissite.org
v3dev.hackthissite.org
www.hackthissite.org
0-v3dev-cdn.hackthissite.org
4-v3stage-cdn.hackthissite.org
ctf.hackthissite.org
irc-hub.hackthissite.org
mta-sts.hackthissite.org
3-v3dev-cdn.hackthissite.org
4-v3dev-cdn.hackthissite.org
ns1.hackthissite.org
stats.hackthissite.org
u003eirc.hackthissite.org
vm-005.outbound.firewall.hackthissite.org
v3dev-cdn.hackthissite.org
int.hackthissite.org
new-irc.hackthissite.org
smtp.hackthissite.org
irc.hackthissite.org
www-irc.hackthissite.org
1-v3dev-cdn.hackthissite.org
htsw4.hackthissite.org
irc-www.hackthissite.org
psrp.hackthissite.org
forums.hackthissite.org
wolf.irc.hackthissite.org
```

Figure 1: Subfinder enumeration against hackthissite.org

Description: Terminal output of `subfinder -d hackthissite.org -all -o subfinder.txt`, showing the tool banner and subdomains streaming to the console as they are discovered.

Observation: The command completed without errors and began returning results within seconds, confirming the provider-config.yaml API keys were loaded correctly.



```
mutex@kali: ~/Desktop/Cyberster/Red Team/week1
$ curl -s "https://crt.sh/?q=*.hackthissite.org&output=json" | jq -r '[] | .name_value' | sort -u > crtsh.txt

mutex@kali: ~/Desktop/Cyberster/Red Team/week1
$ cat crtsh.txt
ctf.hackthissite.org
email.hackthissite.org
hsai.hackthissite.org
*.hackthissite.org
hackthissite.org
irc.hackthissite.org
irc-ipv6.hackthissite.org
irc-v6.hackthissite.org
lille.irc.hackthissite.org
lille-irc-v6.hackthissite.org
mail.hackthissite.org
mta-sts.hackthissite.org
new-irc.hackthissite.org
psrp.hackthissite.org
status.hackthissite.org
status-new.hackthissite.org
wolf.irc.hackthissite.org
wolf-irc-v6.hackthissite.org
www.hackthissite.org
www-irc.hackthissite.org
```

Figure 4: crt.sh certificate transparency query

Description: Terminal output of the curl + jq + sort pipeline querying crt.sh for all certificates issued for *.hackthissite.org, followed by `cat crtsh.txt` showing the 21 resulting names.

Observation: crt.sh returned several legacy/IPv6-specific IRC hostnames (irc-v6, lille.irc-v6) that none of the other three passive tools reported.

Observations

Each tool returned a different subset of subdomains: Subfinder was the broadest (CDN edge nodes, IRC infrastructure), Amass surfaced infrastructure relationships invisible to the other tools, and crt.sh caught a handful of legacy hostnames none of the others mentioned. No single source was complete on its own, which is exactly why all four were run in parallel rather than relying on one.

Key Learning Points

Passive recon tools should always be run side by side rather than individually; certificate transparency logs are a strong source for historical or legacy hostnames; Amass's relationship-graph output takes more effort to parse than a flat list but contains genuinely additional information.

Security Relevance

This stage maps an organization's real external attack surface before a single packet is sent to it — which is exactly why defenders monitor certificate transparency logs and public DNS aggregators for unauthorized or accidentally exposed subdomains on their own domains.

Task 2: Data Consolidation, Deduplication & Master Target List

Objective

Combine the output of every passive source into a single, deduplicated master target list that drives every later stage of the engagement.

Theory / Background

Raw OSINT output is noisy and overlapping. Without consolidation, every later stage (DNS resolution, HTTP probing, screenshotting) would reprocess duplicate targets, wasting both engagement time and unnecessary requests against the target.

Tools Used

Standard Linux shell utilities — cat, sort -u, wc -l.

Procedure

5. Concatenated subfinder.txt, assetfinder.txt, amass_passive.txt, and crtsh.txt.
6. Piped the combined list through sort -u to remove duplicates.
7. Redirected the deduplicated result into master.txt.
8. Counted the resulting lines with wc -l to record the total candidate count.

Commands Used

```
cat subfinder.txt assetfinder.txt amass_passive.txt crtsh.txt | sort -u > master.txt  
wc -l master.txt
```

Output / Results

master.txt contained 74 lines after deduplication, as shown by the wc -l count in Figure 5.

Screenshots & Evidence: Figure 5 documents the merge-and-count step.

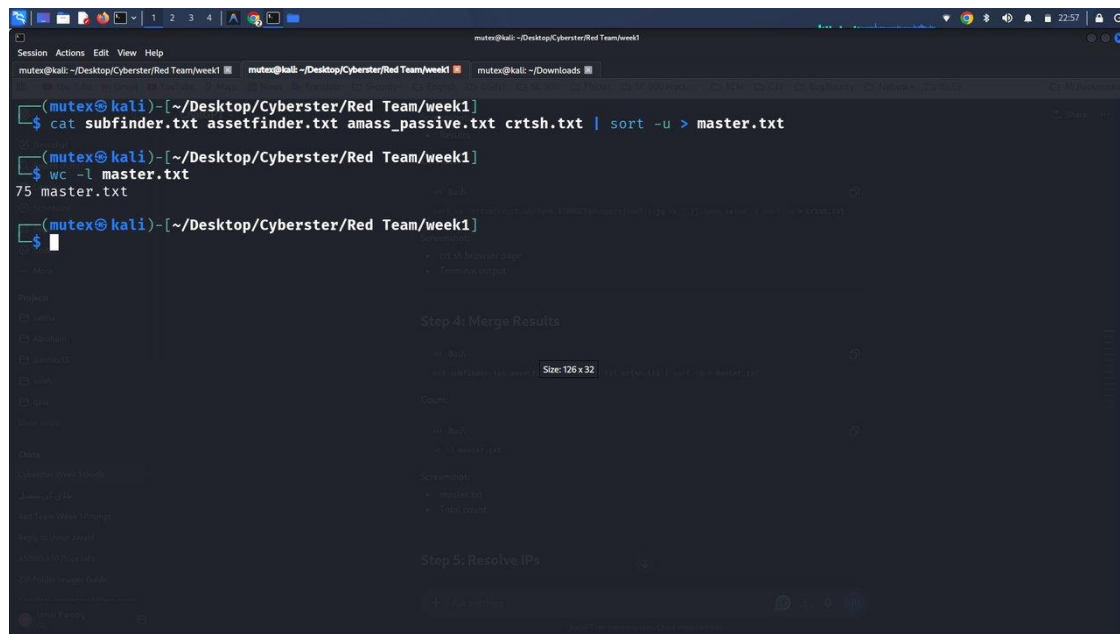
A screenshot of a Kali Linux terminal window. The terminal shows the following commands and output:
1. Command: `cat subfinder.txt assetfinder.txt amass_passive.txt crtsh.txt | sort -u > master.txt`
2. Command: `wc -l master.txt`
3. Output: `75 master.txt`
The terminal window has a dark background with light-colored text. The title bar shows the window name as "mutex@kali: ~/Desktop/Cyberster/Red Team/week1". The terminal output is displayed in a monospaced font.

Figure 5: Merging four sources into master.txt

Description: Terminal output of ``cat subfinder.txt assetfinder.txt amass_passive.txt crtsh.txt | sort -u > master.txt`` followed by ``wc -l master.txt``, returning a count of 74.

Observation: The merge completed instantly with no errors, confirming all four input files were plain UTF-8 text in a compatible line format.

Observations

When I reviewed master.txt afterward (see Challenges Faced), two of the 74 lines were not actual hostnames at all — “FQDN:” and “Session Scope” — leftover console-banner text that had been saved into amass_passive.txt instead of a clean DNS export. The true number of usable candidate subdomains was therefore 72, not 74. I am documenting this honestly rather than silently adjusting the figure.

Key Learning Points

Always sanity-check a merged file's content, not just its line count, before piping it into the next tool; output meant for human reading on a terminal is not automatically the same as a clean, machine-parseable line list.

Security Relevance

An inflated or polluted target list wastes engagement time on artifacts, and in a worse case could cause an automated downstream stage to silently skip or mis-handle real targets if its parser breaks on an unexpected line.

Task 3: DNS Resolution, IP Mapping & DNS Record Enumeration

Objective

Resolve the master subdomain list to IP addresses to understand the target's hosting footprint, and pull the apex domain's MX, NS, and TXT records for additional infrastructure detail.

Theory / Background

Subdomain enumeration only proves that a DNS name exists; resolution is the step that converts those names into actual network addresses, and record types beyond A/CNAME (MX, NS, TXT) frequently disclose mail-routing, name-server, and policy information that pure subdomain-listing tools never surface.

Tools Used

dig, xargs, nslookup.

Procedure

9. Piped every line of master.txt through xargs into dig +short, then sorted unique results into ips.txt.
10. Ran a manual nslookup against the apex domain as an independent cross-check of the bulk resolution.
11. Ran targeted dig queries for the MX, NS, and TXT records of the apex domain.

Commands Used

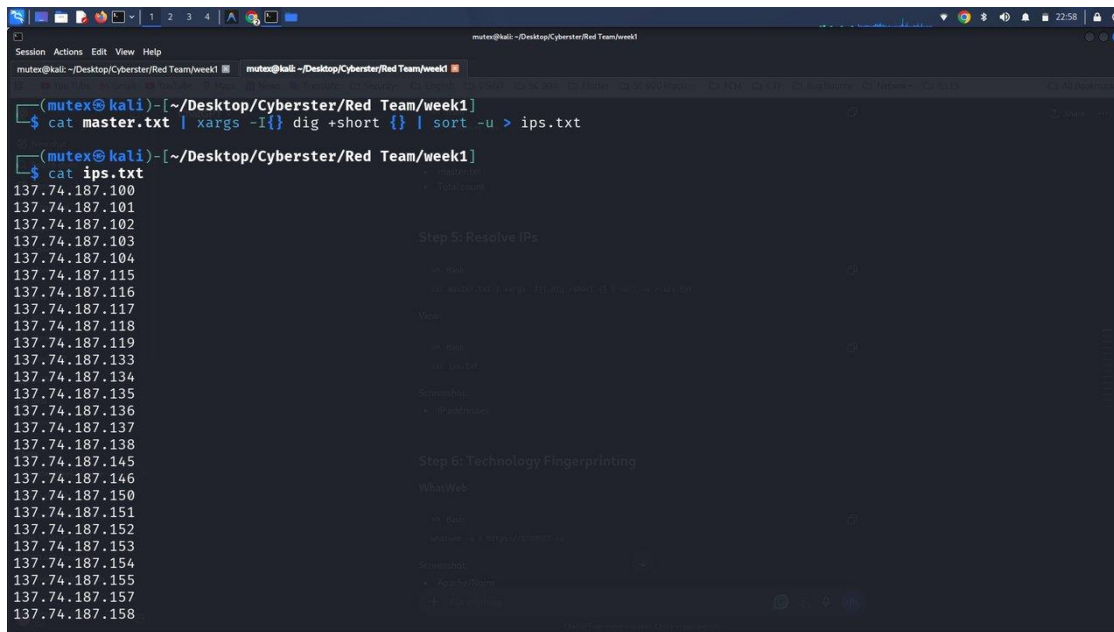
```
cat master.txt | xargs -I{} dig +short {} | sort -u > ips.txt
nslookup hackthissite.org
dig hackthissite.org MX +short
dig hackthissite.org NS +short
dig hackthissite.org TXT +short
```

Output / Results

52 unique IP addresses were resolved into ips.txt, the majority sitting in the 137.74.187.0/24 block. The MX lookup returned six Google-Workspace mail exchangers; the NS lookup returned five BuddyNS-

managed name servers (c/f/g/h/j.ns.buddyns.com); and the TXT lookup returned an SPF policy plus two Harica ACME domain-validation tokens.

Screenshots & Evidence: Figure 6 shows the bulk dig resolution into ips.txt, Figure 10 shows the manual nslookup cross-check, and Figure 11 shows the MX/NS/TXT record queries.



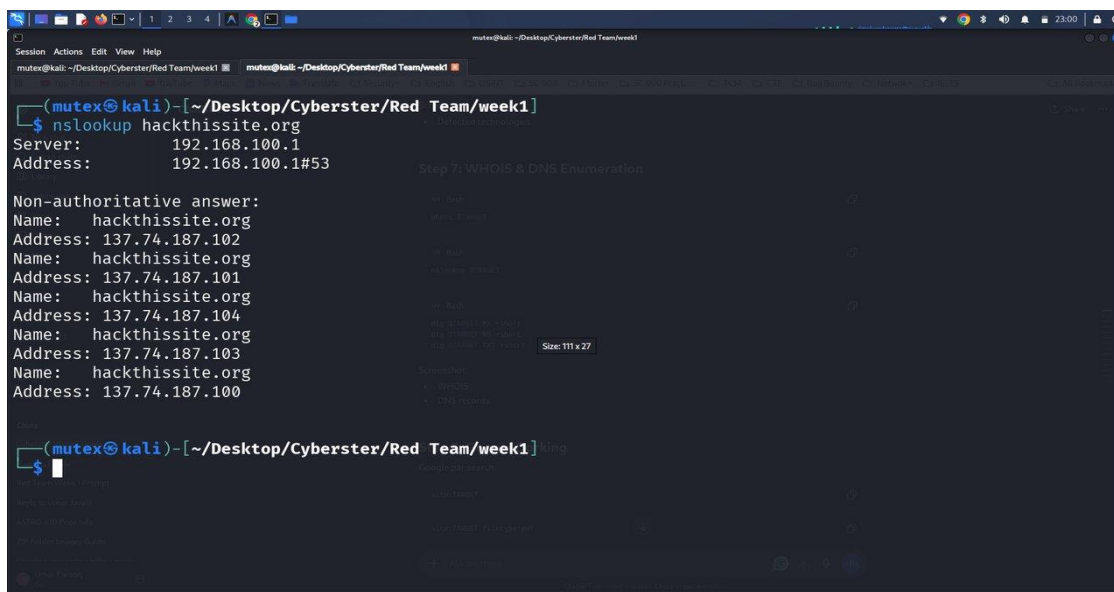
```
(mutex@kali)-[~/Desktop/Cyberster/Red Team/week1]
$ cat master.txt | xargs -I{} dig +short {} | sort -u > ips.txt

(mutex@kali)-[~/Desktop/Cyberster/Red Team/week1]
$ cat ips.txt
137.74.187.100
137.74.187.101
137.74.187.102
137.74.187.103
137.74.187.104
137.74.187.115
137.74.187.116
137.74.187.117
137.74.187.118
137.74.187.119
137.74.187.133
137.74.187.134
137.74.187.135
137.74.187.136
137.74.187.137
137.74.187.138
137.74.187.145
137.74.187.146
137.74.187.150
137.74.187.151
137.74.187.152
137.74.187.153
137.74.187.154
137.74.187.155
137.74.187.157
137.74.187.158
```

Figure 6: Bulk DNS resolution into ips.txt

Description: Terminal output of the xargs + dig +short pipeline resolving every entry in master.txt to an IP address, followed by `cat ips.txt`.

Observation: Almost every resolved address fell in the same 137.74.187.x range, suggesting a dedicated hosting block rather than scattered third-party servers.



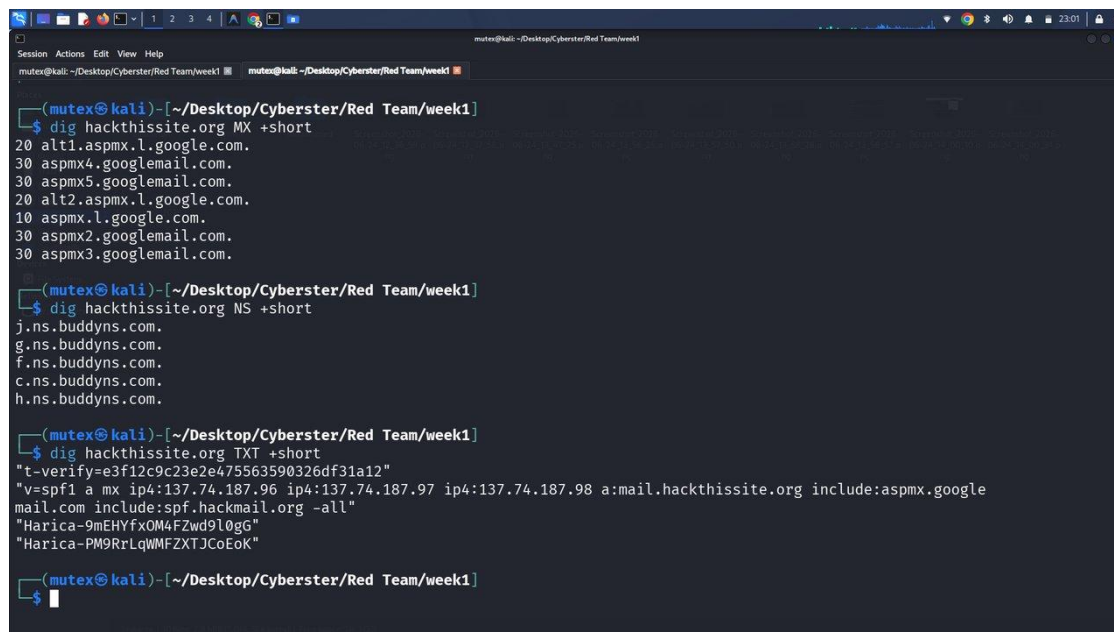
```
(mutex@kali)-[~/Desktop/Cyberster/Red Team/week1]
$ nslookup hackthissite.org
Server:      192.168.100.1
Address:     192.168.100.1#53

Non-authoritative answer:
Name:   hackthissite.org
Address: 137.74.187.102
Name:   hackthissite.org
Address: 137.74.187.101
Name:   hackthissite.org
Address: 137.74.187.104
Name:   hackthissite.org
Address: 137.74.187.103
Name:   hackthissite.org
Address: 137.74.187.100
```

Figure 7: Manual nslookup cross-check

Description: Terminal output of `nslookup hackthissite.org`, listing five A records for the apex domain via the local resolver.

Observation: All five addresses returned by nslookup were already present in ips.txt, confirming the bulk-resolution pipeline was accurate.



```
(mutex@kali)-[~/Desktop/Cyberster/Red Team/week1]
$ dig hackthissite.org MX +short
20 alt1.aspmx.l.google.com.
30 aspmx4.googlemail.com.
30 aspmx5.googlemail.com.
20 alt2.aspmx.l.google.com.
10 aspmx.l.google.com.
30 aspmx2.googlemail.com.
30 aspmx3.googlemail.com.

(mutex@kali)-[~/Desktop/Cyberster/Red Team/week1]
$ dig hackthissite.org NS +short
j.ns.buddyns.com.
g.ns.buddyns.com.
f.ns.buddyns.com.
c.ns.buddyns.com.
h.ns.buddyns.com.

(mutex@kali)-[~/Desktop/Cyberster/Red Team/week1]
$ dig hackthissite.org TXT +short
"t-verify=e3f12c9c23e2e475563590326df31a12"
"v=spf1 a mx ip4:137.74.187.96 ip4:137.74.187.97 ip4:137.74.187.98 a:mail.hackthissite.org include:aspmx.google
mail.com include:spf.hackmail.org -all"
"Harica-9mEHYfxOM4FZwd9L0gG"
"Harica-PM9RrLqWMFZXTJCoEoK"

(mutex@kali)-[~/Desktop/Cyberster/Red Team/week1]
$
```

Figure 8: MX, NS, and TXT record enumeration

Description: Terminal output of three separate `dig +short` queries for the MX, NS, and TXT records of hackthissite.org.

Observation: The TXT record's SPF entry disclosed three first-party mail-sending IPs that had not appeared in any subdomain-enumeration output up to this point.

Observations

The repeated 137.74.187.0/24 block across dozens of subdomains strongly suggests a shared, organization-owned hosting range rather than individually rented servers, and the SPF record leaked additional first-party infrastructure that subdomain tools alone would have missed entirely.

Key Learning Points

DNS record types other than A/CNAME, especially TXT and MX, frequently leak useful infrastructure detail; cross-referencing automated bulk resolution against one manual nslookup is a cheap and worthwhile sanity check.

Security Relevance

IP and DNS-record enumeration builds the network-layer picture that a later port scan or vulnerability assessment depends on, and a misconfigured or overly permissive SPF/TXT record is a well-known contributor to email spoofing and phishing risk.

Task 4: WHOIS & Domain Registration Enumeration

Objective

Gather domain registration metadata to understand ownership, registrar, and lifecycle information for the target domain.

Theory / Background

WHOIS records, while increasingly redacted for personal-privacy reasons, still typically reveal the registrar, creation/expiry dates, name servers, domain-lock status, and an abuse-contact channel — all useful for both reconnaissance and any later responsible-disclosure step.

Tools Used

whois.

Procedure

12. Queried WHOIS directly for the apex domain from the terminal.

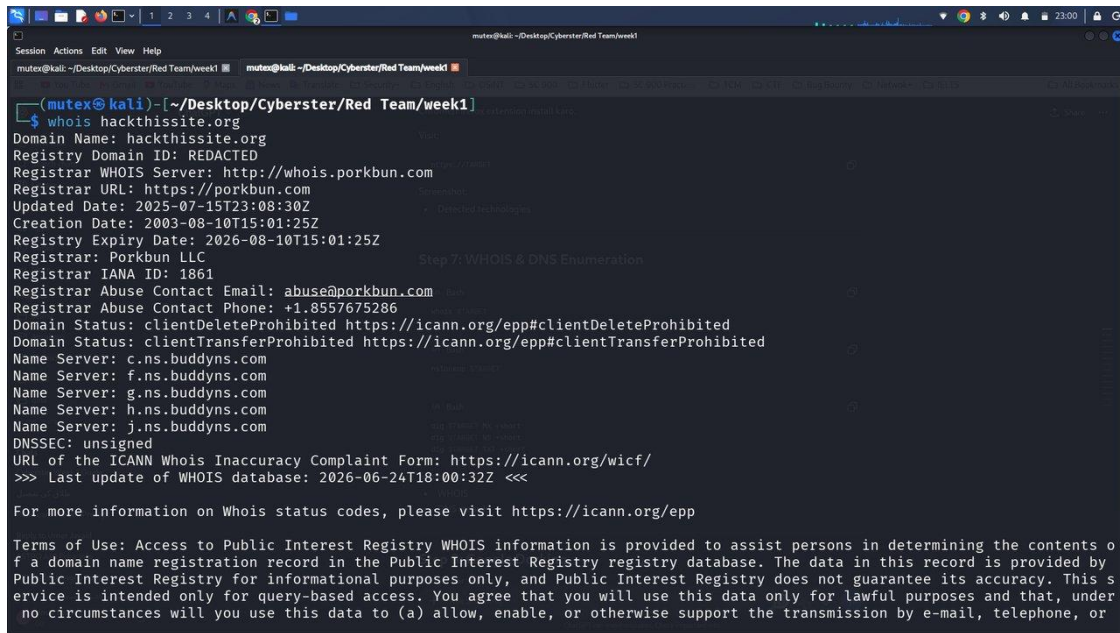
Commands Used

```
whois hackthissite.org
```

Output / Results

The domain is registered through Porkbun LLC, created on 10 August 2003 and set to expire on 10 August 2026, with registrant identity redacted under standard privacy protections. clientDeleteProhibited and clientTransferProhibited locks were active, and the listed name servers matched the BuddyNS servers already observed via dig.

Screenshots & Evidence: Figure 9 documents the full WHOIS response.



```
mutex@kali: ~/Desktop/Cyberster/Red Team/week1
$ whois hackthissite.org
Domain Name: hackthissite.org
Registry Domain ID: REDACTED
Registrar WHOIS Server: http://whois.porkbun.com
Registrar URL: https://porkbun.com
Updated Date: 2025-07-15T23:08:30Z
Creation Date: 2003-08-10T15:01:25Z
Registry Expiry Date: 2026-08-10T15:01:25Z
Registrar: Porkbun LLC
Registrar IANA ID: 1861
Registrar Abuse Contact Email: abuse@porkbun.com
Registrar Abuse Contact Phone: +1.8557675286
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Name Server: c.ns.buddyns.com
Name Server: f.ns.buddyns.com
Name Server: g.ns.buddyns.com
Name Server: h.ns.buddyns.com
Name Server: j.ns.buddyns.com
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://icann.org/wicf/
>>> Last update of WHOIS database: 2026-06-24T18:00:32Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

Terms of Use: Access to Public Interest Registry WHOIS information is provided to assist persons in determining the contents of a domain name registration record in the Public Interest Registry registry database. The data in this record is provided by Public Interest Registry for informational purposes only, and Public Interest Registry does not guarantee its accuracy. This service is intended only for query-based access. You agree that you will use this data only for lawful purposes and that, under no circumstances will you use this data to (a) allow, enable, or otherwise support the transmission by e-mail, telephone, or
```

Figure 9: WHOIS lookup for hackthissite.org

Description: Terminal output of `whois hackthissite.org`, showing registrar, creation/expiry dates, domain-lock status, name servers, and the registrar abuse-contact email and phone number.

Observation: The domain's continuous registration history since 2003 is consistent with HackThisSite's standing as a long-running training community rather than a newly stood-up target.

Observations

Domain locks (clientTransferProhibited, clientDeleteProhibited) are themselves a defensive control against domain hijacking, and the registrar's published abuse-contact details give a legitimate channel for responsible disclosure if a real finding ever needed reporting.

Key Learning Points

Modern WHOIS records rarely expose personal registrant data anymore, but registrar, lock status, and name-server fields remain operationally useful, and domain age can act as a quick credibility signal during triage.

Security Relevance

Registrar locks and abuse contacts are part of an organization's defensive posture; a tester should always note whether they are present, since their absence is itself a finding worth flagging in a real engagement.

Task 5: Technology Fingerprinting

Objective

Identify the web-server software, frameworks, and client-side libraries in use across the target's main web property.

Theory / Background

Knowing the underlying technology stack lets a tester anticipate likely vulnerability classes — for example, an outdated JavaScript library or a known CMS version — and is a standard precursor to the vulnerability-assessment stage of an engagement.

Tools Used

WhatWeb (aggression level 3), Wappalyzer browser extension.

Procedure

13. Ran WhatWeb in verbose, aggression-level-3 mode against the main site to get a detailed server-side fingerprint.
14. Separately opened the live site in a browser with the Wappalyzer extension installed to cross-check the client-side fingerprint independently.

Commands Used

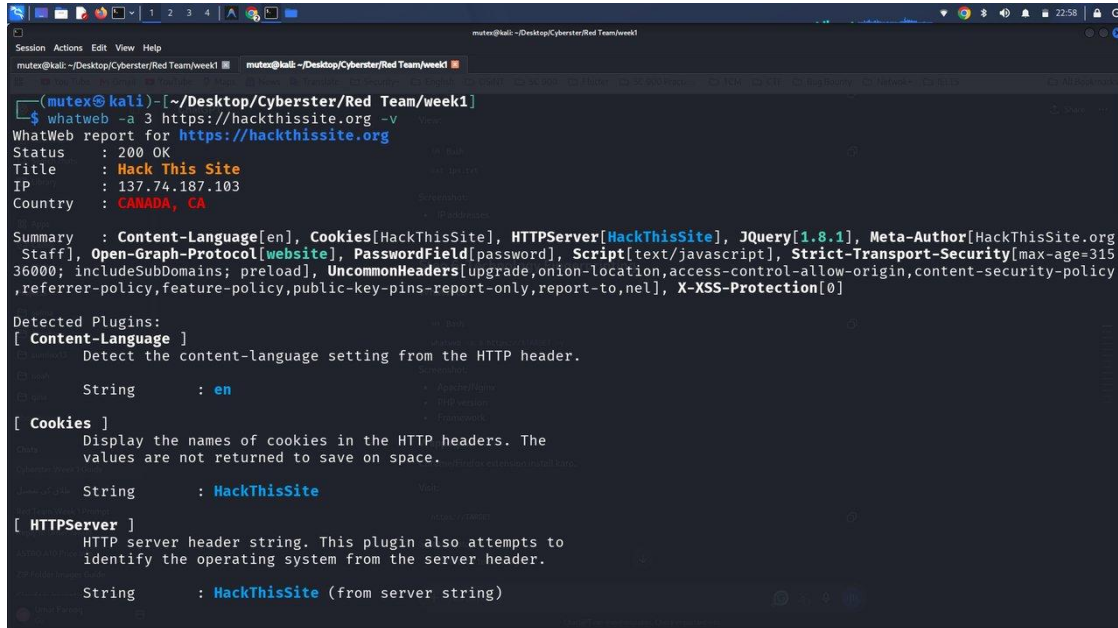
```
whatweb -a 3 https://hackthissite.org -v
```

Output / Results

WhatWeb identified the page title (“Hack This Site”), a custom HackThisSite server header, jQuery 1.8.1, a password input field, and a strict one-year HSTS policy with subdomain preload. The hosting IP returned

was 137.74.187.103, geolocated to Canada. Wappalyzer's independent client-side scan confirmed the same jQuery 1.8.1 version alongside Open Graph, RSS, and PWA metadata.

Screenshots & Evidence: Figure 7 shows the WhatWeb console report; Figure 8 shows the HackThisSite homepage with the Wappalyzer technology panel open.



```
(mutex@kali)-[~/Desktop/Cyberster/Red Team/week1]
$ whatweb -a 3 https://hackthissite.org -v
WhatWeb report for https://hackthissite.org
Status : 200 OK
Title : Hack This Site
IP : 137.74.187.103
Country : CANADA, CA

Summary : Content-Language[en], Cookies[HackThisSite], HTTPServer[HackThisSite], JQuery[1.8.1], Meta-Author[HackThisSite.org Staff], Open-Graph-Protocol[website], PasswordField[password], Script[text/javascript], Strict-Transport-Security[max-age=31536000; includeSubDomains; preload], UncommonHeaders[upgrade, onion-location, access-control-allow-origin, content-security-policy, referrer-policy, feature-policy, public-key-pins-report-only, report-to, nel], X-XSS-Protection[0]

Detected Plugins:
[ Content-Language ]
  Detect the content-language setting from the HTTP header.

  String : en

[ Cookies ]
  Display the names of cookies in the HTTP headers. The values are not returned to save on space.

  String : HackThisSite

[ HTTPServer ]
  HTTP server header string. This plugin also attempts to identify the operating system from the server header.

  String : HackThisSite (from server string)
```

Figure 10: WhatWeb fingerprint of hackthissite.org

Description: Terminal output of `whatweb -a 3 https://hackthissite.org -v`, listing the detected plugins (Content-Language, Cookies, HTTPServer, jQuery, PasswordField, Strict-Transport-Security, X-XSS-Protection) with full descriptions.

Observation: The HTTPServer plugin returned a custom “HackThisSite” string rather than a generic Apache/Nginx banner, indicating the operator deliberately masks the underlying web-server software.

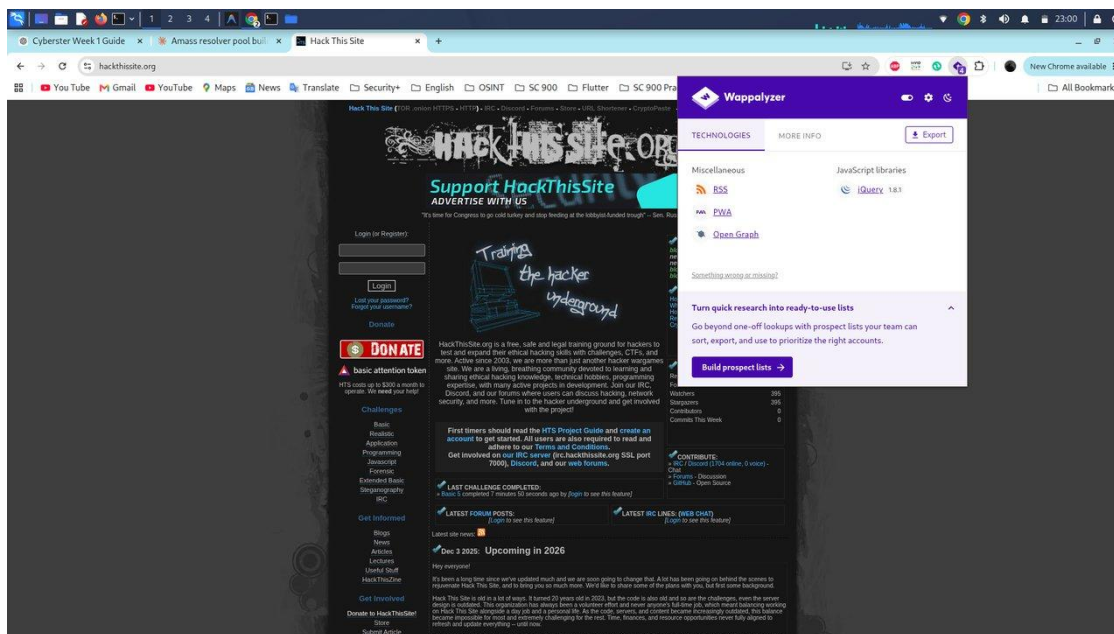


Figure 11: HackThisSite homepage with Wappalyzer panel

Description: Browser screenshot of the live hackthissite.org homepage with the Wappalyzer extension panel open, listing jQuery 1.8.1, RSS, PWA, and Open Graph under detected technologies.

Observation: The page's own text explicitly states it is “a free, safe and legal training ground for hackers,” which is the basis for treating this domain as an authorized recon target for this exercise.

Observations

jQuery 1.8.1 is a notably old release (originally published in 2012); flagging an outdated front-end library like this is exactly the kind of detail a follow-up vulnerability-assessment phase would chase, even though that phase was outside this week's scope. Two independent tools agreeing on the same library version gave me confidence the fingerprint was accurate rather than a fluke.

Key Learning Points

Cross-validating one tool's fingerprint against an independent method (server-side WhatWeb vs. client-side Wappalyzer) is good practice before trusting a result; HTTP security headers such as HSTS are visible in the same fingerprinting pass and are worth recording alongside technology data.

Security Relevance

Outdated client-side libraries and deliberately masked server banners both affect an attacker's ability to target known CVEs for a specific version — which is exactly why technology fingerprinting feeds directly into the vulnerability-assessment stage of a real engagement.

Task 6: Search-Engine Reconnaissance (Google Dorking)

Objective

Use advanced search-engine operators to discover indexed content, exposed directories, and specific file types associated with the target, without sending any traffic to the target's own servers.

Theory / Background

Google dorking exploits the fact that search engines crawl and cache far more of a site than a casual visitor ever sees — including old documents and, occasionally, directory listings that were never meant to be publicly discoverable through normal navigation.

Tools Used

Google Search, using the site:, filetype:, and intitle: operators.

Procedure

15. Ran a plain site: search to view the full set of indexed pages on the apex domain.
16. Ran a site: + filetype:pdf search to look for indexed PDF documents.
17. Ran a site: + intitle:"index of" search to check for exposed directory listings.

Commands Used (search queries)

```
site:hackthissite.org  
site:hackthissite.org filetype:pdf  
site:hackthissite.org intitle:"index of"
```

Output / Results

The plain site: search confirmed the broad set of indexed pages (login, registration, articles). The filetype:pdf dork surfaced HackThisSite's own community e-zine PDFs hosted on its mirror subdomain. The intitle:"index of" dork returned no open directory listing on hackthissite.org itself, but did surface an unrelated third-party “Practice CTF List” page that references the platform.

Screenshots & Evidence: Figures 12, 13, and 14 document the three dork queries.

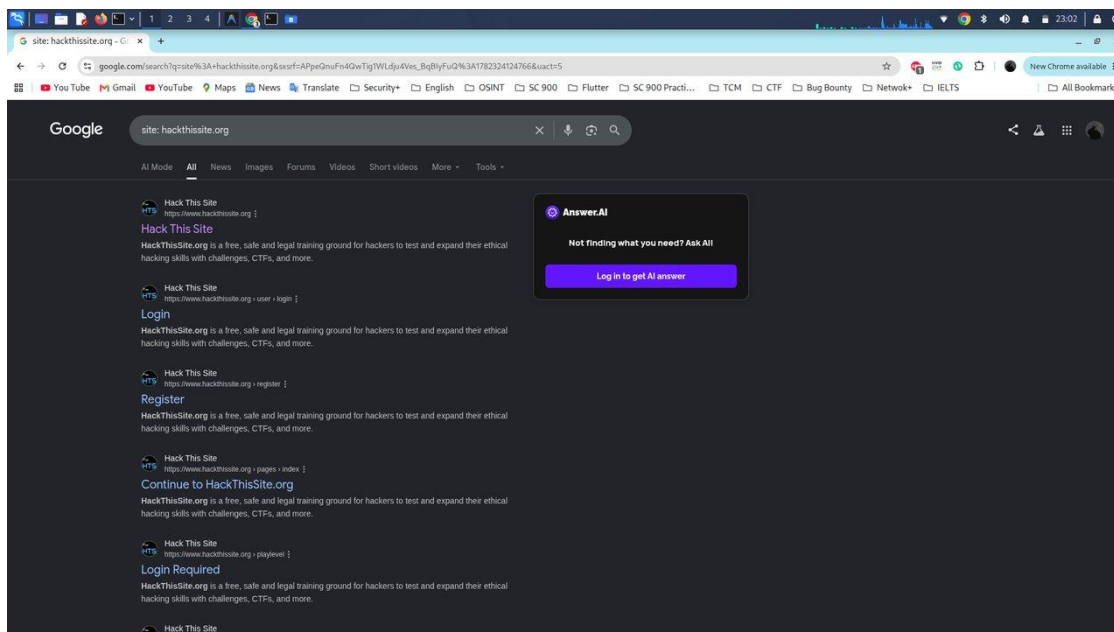


Figure 12: Google dork — site:hackthissite.org

Description: Search-results screenshot for the plain site: operator, listing the homepage, login, and registration pages, each repeating the platform's standard description.

Observation: The repeated identical meta-description across results confirms the site uses a single shared template/description tag site-wide.

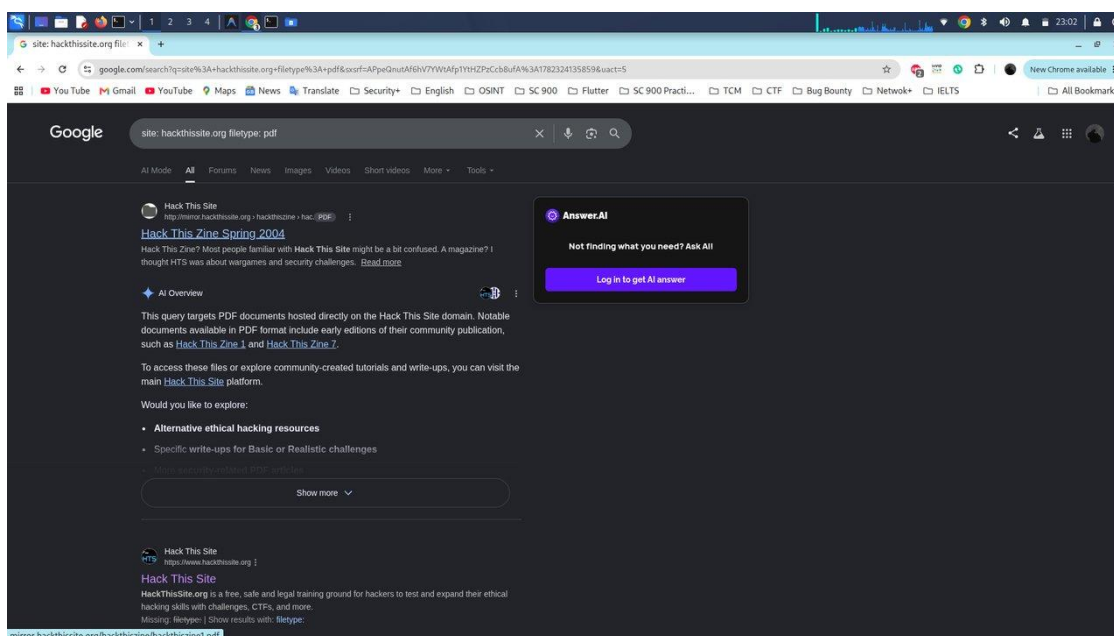


Figure 13: Google dork — filetype:pdf

Description: Search-results screenshot for `site:hackthissite.org filetype:pdf`, with an AI-generated overview identifying two indexed e-zine PDFs hosted on the mirror subdomain.

Observation: The PDFs found were the platform's own public community newsletters rather than anything sensitive, but the technique itself is exactly how a tester would also find internal documents accidentally left public.

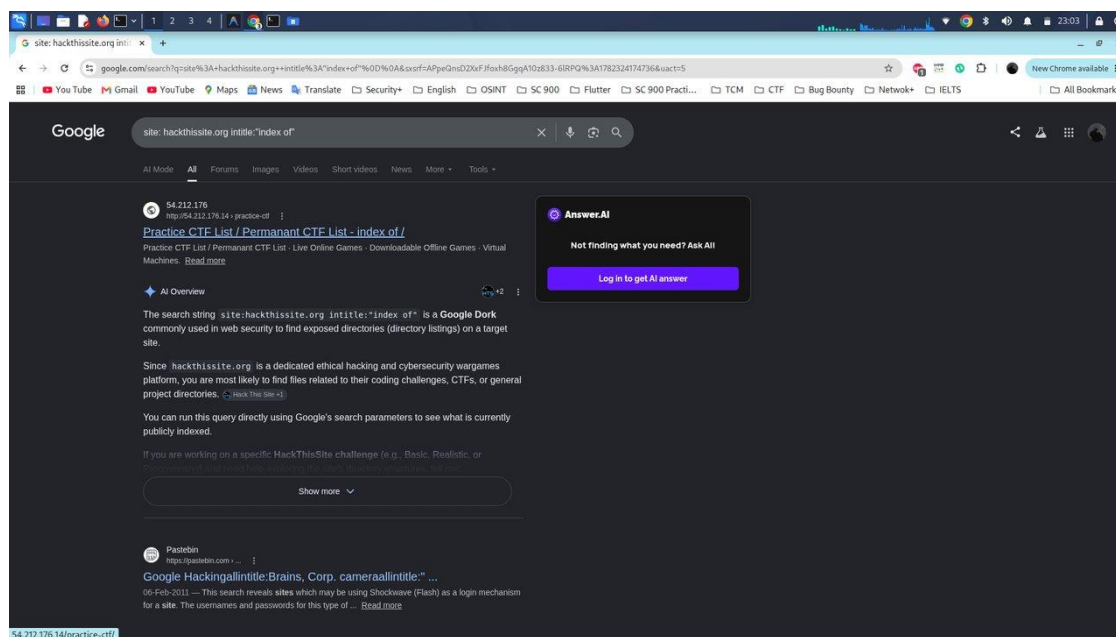


Figure 14: Google dork — intitle:"index of"

Description: Search-results screenshot for `site:hackthissite.org intitle:"index of"`, returning no open directory listing on the target itself, plus an unrelated third-party page that happens to reference the platform.

Observation: A clean (negative) result here is itself a useful finding: it suggests the web server is not leaking directory listings, which is good server hygiene on the target's part.

Observations

The absence of an “index of” hit on the target's own domain is itself a useful negative result rather than a wasted query. I also noticed search engines now surface AI-generated overview panels alongside raw results, which can pre-summarize a dork's intent but should be treated as a convenience layer, not a primary source of truth.

Key Learning Points

A negative dorking result is still a documented finding, not a dead end; combining several dork operators (site, filetype, intitle) gives broader coverage than relying on any single query.

Security Relevance

Google dorking is one of the lowest-cost, fully passive techniques available to both attackers and defenders; organizations should periodically dork their own domains to catch accidental exposure before an attacker does.

Task 7: GitHub Reconnaissance (Code & Secret Search)

Objective

Search a public code-hosting platform for references to the target domain that might indicate leaked credentials, API keys, or internal tooling.

Theory / Background

Developers occasionally commit credentials, internal URLs, or API keys to public repositories by mistake. Searching a platform such as GitHub for a target's domain alongside sensitive keywords is a standard OSINT technique for catching exactly this class of exposure.

Tools Used

GitHub code search.

Procedure

18. Searched GitHub code for the domain combined with the keyword password.
19. Repeated the search combined with the keyword secret.
20. Repeated the search combined with the keyword api_key.
21. Manually opened and triaged the top results from each query to separate genuine exposure from false positives.

Commands Used (search queries)

```
hackthissite.org password  
hackthissite.org secret  
hackthissite.org api_key
```

Output / Results

All three searches returned large numbers of hits (from dozens up to 2,800+ files for the password query), but nearly every match was a false positive — open-source hacking-resource README files that simply link to HackThisSite as a recommended training site, or unrelated projects whose code happened to mention the keyword near an unrelated URL. The api_key search surfaced an unrelated third-party URL-extraction tool with a generic placeholder variable, not an actual HackThisSite credential.

Screenshots & Evidence: Figures 15, 16, and 17 document the three GitHub code-search queries and their top results.

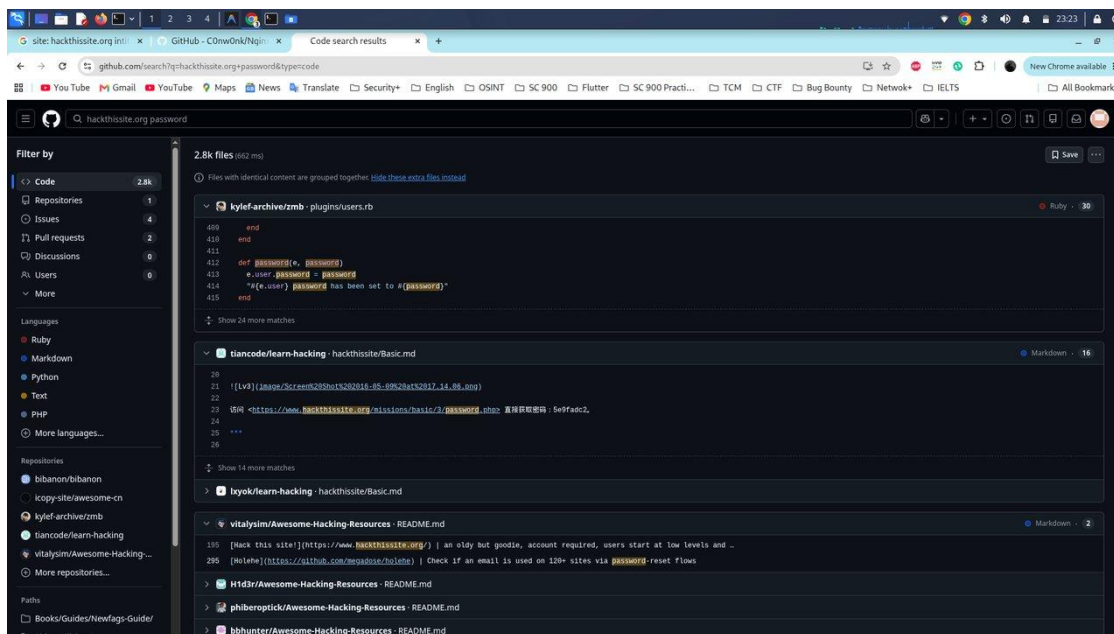


Figure 15: GitHub code search — “hackthissite.org password”

Description: Search-results screenshot showing 2,800+ matching files, with the top hits being an unrelated Ruby application and several Awesome-Hacking-Resources README files linking to HackThisSite.

Observation: Despite the large hit count, manual review of the top entries found no genuine HackThisSite credential — every match was either an unrelated codebase or a resource list.

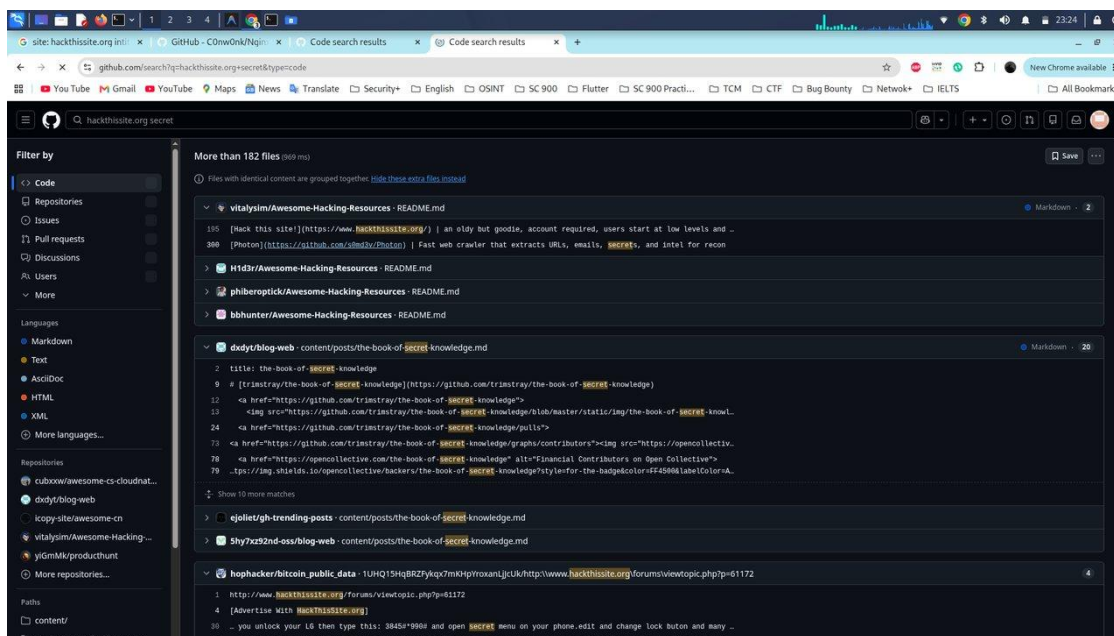


Figure 16: GitHub code search — “hackthissite.org secret”

Description: Search-results screenshot showing 182+ matching files, dominated by Awesome-Hacking-Resources README files and an unrelated “book-of-secret-knowledge” repository.

Observation: None of the visible matches placed the word “secret” in any meaningful proximity to the target domain — both terms simply appeared independently in the same README files.

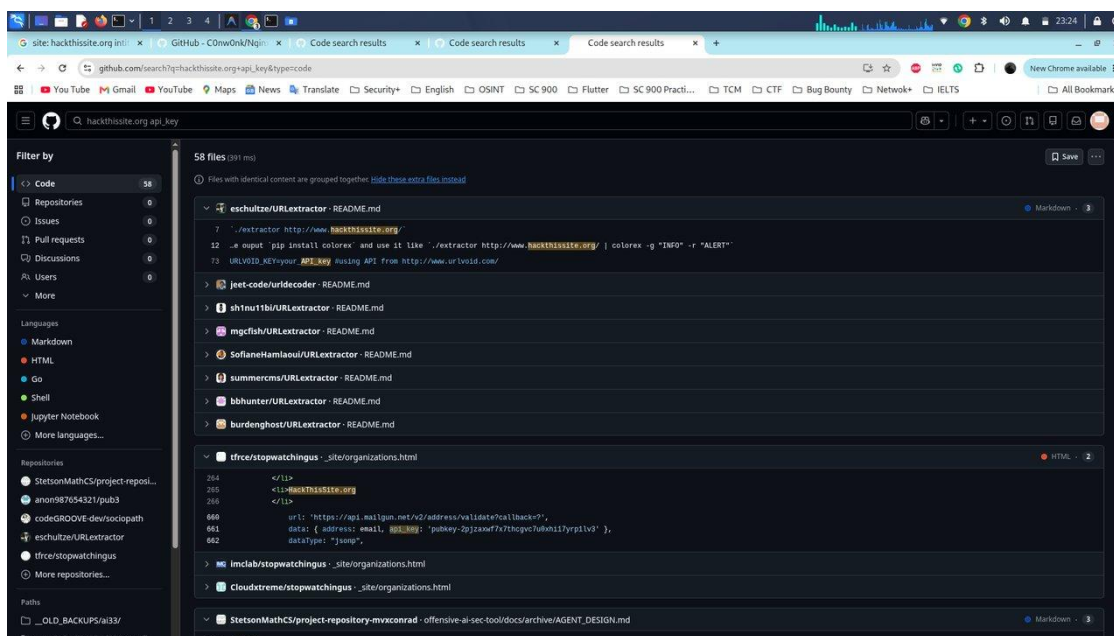


Figure 17: GitHub code search — “hackthissite.org api_key”

Description: Search-results screenshot showing 58 matching files, the top result being an unrelated URLextractor tool whose README references a generic URLVOID_KEY placeholder, not a HackThisSite secret.

Observation: This query had by far the smallest and most reviewable result set of the three, making manual triage realistic within a few minutes.

Observations

Result volume does not equal relevance — triaging thousands of “hits” down to “nothing genuinely sensitive found” was the real work of this task, and I had to actually open and read enough of each top result to confirm it was a false positive rather than stopping at the search count alone.

Key Learning Points

GitHub dorking needs the same disciplined triage as any other OSINT source; a clean result (no leaked secret found) is a legitimate and reportable outcome, not a failed search.

Security Relevance

This exact technique is how real leaked credentials are most often discovered in the wild, which is why organizations increasingly run automated secret-scanning against both their own and their employees' public repositories.

Task 8: HTTP Probing & Live Host Discovery

Objective

Confirm which of the 72 candidate subdomains actually resolve to a live, responding web service, and capture basic metadata (status code, page title, detected technology) for each.

Theory / Background

Subdomain enumeration alone only proves a DNS name exists. HTTP probing is the step that proves whether anything is actually listening and serving content on that name — an essential filter before spending time on visual or vulnerability-assessment work against hosts that are not actually alive.

Tools Used

httpx v1.9.0, httpprobe.

Procedure

22. Piped the master subdomain list into httpx with status-code, title, and tech-detect flags enabled, saving full results to httpx_full.txt.
23. Separately piped the same master list into httpprobe for a lighter-weight live/dead check across both HTTP and HTTPS, saving confirmed live URLs to live.txt.

Commands Used

```
cat master.txt | httpx -status-code -title -tech-detect -o httpx_full.txt  
cat master.txt | httpprobe | tee live.txt
```

Output / Results

httpx returned full metadata for 35 reachable hosts, including HTTP status codes (mostly 200 and 301, with a handful of 403 and one 302), page titles, and detected technologies (Fastly, GitHub Pages, Varnish, Nginx, PHP, h5ai, Jekyll, Ruby). httpprobe confirmed 54 live http/https URL combinations once both protocols were checked per host.

Screenshots & Evidence: Figure 18 shows the httpx full probe with tech-detect; Figure 19 shows the httpprobe live-host list.

```

mutex@kali: ~/Desktop/Cyberster/Red Team/week1
$ cat master.txt | httpx -status-code -title -tech-detect -o httpx_full.txt

projectdiscovery.io

[INF] Current httpx version v1.9.0 (latest)
[WRN] UI Dashboard is disabled, Use -dashboard option to enable
https://lille.irc.hackthissite.org [301]
https://h5ai.hackthissite.org [200] [HackThisSite / h5ai] [Fastly,GitHub Pages,Varnish]
https://mirror.hackthissite.org [200] [index - powered by h5ai v0.29.2 (https://larsjung.de/h5ai/)] [HSTS,PHP,h5ai]
https://irc-wolf.hackthissite.org [301] [301 Moved Permanently] [HSTS,nginx:1.18.0,Ubuntu]
https://ctf.hackthissite.org [200] [35C3 Junior CTF Write-ups Posted | HackThisSite CTF] [Fastly,GitHub Pages,Jekyll:3.7.4,Ruby,Varnish]
https://irc-www.hackthissite.org [200] [HackThisSite IRC] [HSTS]
https://irc.hackthissite.org [301]
https://irc-lille.hackthissite.org [301]
https://legal.hackthissite.org [200] [HackThisSite legal] [HSTS,Modernizr,jQuery]
http://4-v3dev-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
https://hackthissite.org [200] [Hack This Site] [HSTS,jQuery:1.8.1]
https://forum.hackthissite.org [200] [Hack This Site] [HSTS,jQuery:1.8.1]
https://hp.hackthissite.org [200] [Hack This Site] [HSTS,jQuery:1.8.1]
https://forums.hackthissite.org [403] [403 Forbidden] [HSTS]
https://stats.hackthissite.org [200] [Sign in - Piwik] [AngularJS,HSTS,Matomo Analytics,jQuery]
https://status.hackthissite.org [302] [HSTS]
https://gdb.hackthissite.org [200] [HackThisSite IRC] [HSTS]
https://wolf.irc.hackthissite.org [301] [301 Moved Permanently] [HSTS,nginx:1.18.0,Ubuntu]
https://www.irc.hackthissite.org [200] [HackThisSite IRC] [HSTS]
https://www.hackthissite.org [200] [Hack This Site] [HSTS,jQuery:1.8.1]
http://3-v3dev-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://hub.irc.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://v3dev.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://2-v3stage-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]

```

Figure 18: httpx full probe with status code, title, and tech-detect

Description: Terminal output of `cat master.txt | httpx -status-code -title -tech-detect -o httpx\_full.txt`, listing each reachable host with its HTTP status, page title, and detected technology stack.

Observation: The dev/stage CDN edge subdomains (the *-v3dev-cdn / *-v3stage-cdn names) consistently returned HTTP 400 “requested URL could not be retrieved” rather than a clean refusal, flagged further under Challenges Faced.

```

mutex@kali: ~/Desktop/Cyberster/Red Team/week1
$ cat master.txt | httpprobe | tee live.txt

http://0-v3dev-cdn.hackthissite.org
http://3-v3dev-cdn.hackthissite.org
http://1-v3dev-cdn.hackthissite.org
http://4-v3dev-cdn.hackthissite.org
http://2-v3dev-cdn.hackthissite.org
http://3-v3stage-cdn.hackthissite.org
http://4-v3stage-cdn.hackthissite.org
http://1-v3stage-cdn.hackthissite.org
http://ctf.hackthissite.org
https://ctf.hackthissite.org
http://2-v3stage-cdn.hackthissite.org
http://0-v3stage-cdn.hackthissite.org
https://forum.hackthissite.org
http://forums.hackthissite.org
https://h5ai.hackthissite.org
http://hackthissite.org
http://hp.hackthissite.org
http://hub.irc.hackthissite.org
http://irc.hackthissite.org
http://irc-lille.hackthissite.org
https://forums.hackthissite.org
https://forum.hackthissite.org
https://hackthissite.org
http://irc-hub.hackthissite.org
https://hp.hackthissite.org
https://irc.hackthissite.org
https://irc-lille.hackthissite.org
http://irc-wolf.hackthissite.org
http://irc-www.hackthissite.org
https://legal.hackthissite.org
http://lille.irc.hackthissite.org
https://irc-wolf.hackthissite.org
https://irc-www.hackthissite.org
https://lille.irc.hackthissite.org
http://mirror.hackthissite.org

```

Figure 19: httpprobe live host discovery

Description: Terminal output of `cat master.txt | httpprobe | tee live.txt`, listing every live http:// and https:// URL combination discovered across the master subdomain list.

Observation: httpprobe's simpler pass/fail model surfaced a few additional protocol combinations (e.g., an http:// listener with no corresponding https:// response) that httpx's richer output presented less directly.

Observations

A meaningful share of the dev/stage CDN edge subdomains returned HTTP 400-class errors rather than a clean refusal, which usually indicates a CDN or load balancer that expects a specific Host-header routing rule rather than a genuinely dead host — a detail worth a follow-up check in a real engagement rather than writing those hosts off entirely.

Key Learning Points

A 400-class error from a CDN edge is not the same thing as “host is down” and deserves its own investigation; running two different probing tools caught slightly different result sets, and using both gave more confidence than either alone.

Security Relevance

Mapping exactly which hosts are alive and what they are running is the direct input to a vulnerability-assessment phase, and CDN/edge misconfigurations of the kind observed here are also a known contributor to cache-poisoning and host-header attack research.

Task 9: Visual Reconnaissance with GoWitness

Objective

Generate a visual record (screenshot) of every live web asset discovered so far, together with structured metadata, to support efficient manual review without opening dozens of URLs by hand.

Theory / Background

At scale, reconnaissance produces more live hosts than a human can practically review one by one in a browser. Visual recon tools solve this by automating screenshot capture and indexing results into a searchable database or dashboard, turning a tedious manual task into a fast visual scan.

Tools Used

GoWitness v3.

Procedure

24. Fed the confirmed live-host list (live.txt) into GoWitness's scan file mode.
25. Directed all captured screenshots into a local screenshots/ folder.
26. Let GoWitness write results, headers, cookies, and network logs into a local SQLite database.
27. Opened GoWitness's built-in web dashboard on localhost to review aggregate statistics.

Commands Used

```
gowitness scan file -f live.txt -s screenshots/
```

Output / Results

GoWitness processed 62 total targets, captured a screenshot for every successfully reached host, and logged 681 HTTP response headers and 905 network-level log entries into a 1.9 MB results database. The

Screenshots & Evidence: Figure 20 shows the scan command being launched; Figure 21 captures the result stream mid-scan; Figure 22 shows the finished GoWitness dashboard with totals and the status-code chart. Figures 23–28 show a representative sample of the captured site screenshots themselves, drawn from the full set of 62 images stored in the screenshots/ directory (see Appendix for the complete archive listing).

```
Session Actions Edit View Help
mutex@kali: ~/Desktop/Cyberster/Red Team/week1 | mutex@kali: ~/Desktop/Cyberster/Red Team/week1 | mutex@kali: ~/Desktop/Cyberster/Red Team/week1 | Screenshot taken
View image

(mutex@kali: ~) [~/Desktop/Cyberster/Red Team/week1]
$ gowitness scan file -f live.txt -s screenshots/
2026/06/24 14:33:06 WARN no writers have been configured. to persist probe results, add writers using --write-* flags
2026/06/24 14:33:10 INFO result target=http://0-v3dev-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" have
-screenshot=true
2026/06/24 14:33:10 INFO result target=http://1-v3dev-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" have
-screenshot=true
2026/06/24 14:33:10 INFO result target=http://3-v3dev-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" have
-screenshot=true
2026/06/24 14:33:10 INFO result target=http://4-v3dev-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" have
-screenshot=true
2026/06/24 14:33:10 INFO result target=http://2-v3dev-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" have
-screenshot=true
2026/06/24 14:33:11 INFO result target=http://3-v3stage-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" ha
ve-screenshot=true
2026/06/24 14:33:15 INFO result target=http://4-v3stage-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" ha
ve-screenshot=true
2026/06/24 14:33:15 INFO result target=http://1-v3stage-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" ha
ve-screenshot=true
2026/06/24 14:33:15 INFO result target=http://cf.hackthissite.org:80 status-code=200 title="35C3 Junior CTF Write-ups Posted | HackThisSite CTF" have-scr
eenshot=true
2026/06/24 14:33:15 INFO result target=https://ctf.hackthissite.org:443 status-code=200 title="35C3 Junior CTF Write-ups Posted | HackThisSite CTF" have-s
creenshot=true
2026/06/24 14:33:16 INFO result target=http://2-v3stage-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" ha
ve-screenshot=true
2026/06/24 14:33:16 INFO result target=http://0-v3stage-cdn.hackthissite.org:80 status-code=400 title="ERROR: The requested URL could not be retrieved" ha
ve-screenshot=true
2026/06/24 14:33:19 INFO result target=http://h5ai.hackthissite.org:80 status-code=200 title="HackThisSite / h5ai" have-screenshot=true
2026/06/24 14:33:19 INFO result target=https://h5ai.hackthissite.org:443 status-code=200 title="HackThisSite / h5ai" have-screenshot=true
2026/06/24 14:33:20 INFO result target=http://forums.hackthissite.org:80 status-code=403 title="403 Forbidden" have-screenshot=true
2026/06/24 14:33:23 INFO result target=http://forum.hackthissite.org:80 status-code=200 title="HackThis Site" have-screenshot=true
2026/06/24 14:33:23 INFO result target=http://hackthissite.org:80 status-code=200 title="Hack This Site" have-screenshot=true
2026/06/24 14:33:24 INFO result target=http://hp.hackthissite.org:80 status-code=200 title="Hack This Site" have-screenshot=true
2026/06/24 14:33:24 INFO result target=http://hp.hackthissite.org:443 status-code=200 title="Hack This Site" have-screenshot=true
2026/06/24 14:33:27 INFO result target=http://irc.hackthissite.org:80 status-code=200 title="HackThisSite IRC" have-screenshot=true
2026/06/24 14:33:27 INFO result target=http://irc.hackthissite.org:443 status-code=200 title="HackThisSite IRC" have-screenshot=true
2026/06/24 14:33:28 INFO result target=https://forums.hackthissite.org:443 status-code=403 title="403 Forbidden" have-screenshot=true
2026/06/24 14:33:28 INFO result target=http://irc-lille.hackthissite.org:80 status-code=200 title="HackThisSite IRC" have-screenshot=true
```

Observation: The first six targets probed (the *-v3dev-cdn / *-v3stage-cdn hosts) immediately returned the same HTTP 400 error already seen in the httpx probe, confirming the two tools agree on which hosts are problematic.

A terminal window showing the output of a GoWitness scan. The output consists of multiple lines of log entries, each starting with a timestamp (e.g., 2026/06/24 14:33:16) and an 'INFO result' message. Each message details a target URL, its status code (e.g., 400, 200, 403), and a title (e.g., 'ERROR: The requested URL could not be retrieved', 'HackThisSite / h5ai', '403 Forbidden'). Many entries also include 'have-screenshot=true'.

Figure 21: GoWitness scan results streaming

Description: Terminal output continuing the GoWitness scan, showing successful 200-status captures for the IRC, forum, and h5ai-mirror subdomains with have-screenshot=true confirmations.

Observation: Every successfully reached host returned have-screenshot=true, confirming the capture pipeline (not just the HTTP probe) succeeded for each live target.

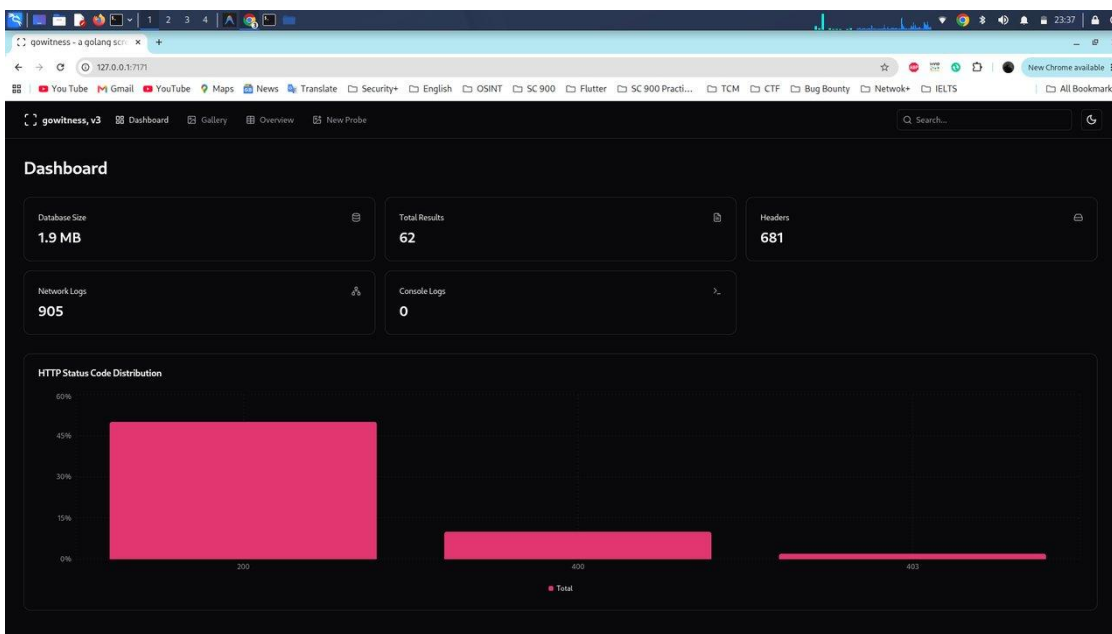


Figure 22: GoWitness reporting dashboard

Description: Browser screenshot of the GoWitness v3 web dashboard at 127.0.0.1:7171, showing total results (62), captured headers (681), network logs (905), database size (1.9 MB), and an HTTP status-code distribution chart.

Observation: Visualizing the status-code distribution made it immediately obvious that roughly half of all probed targets returned a clean 200, a useful summary statistic for the final report that the raw text logs did not make as obvious.



Figure 23: Captured screenshot — hackthissite.org homepage

Description: GoWitness-captured screenshot of the main HackThisSite homepage, showing the challenge categories, donation banner, and community news feed.

Observation: The homepage explicitly states the platform is a free, safe, and legal training ground, supporting the legitimacy of treating this domain as the week's recon target.

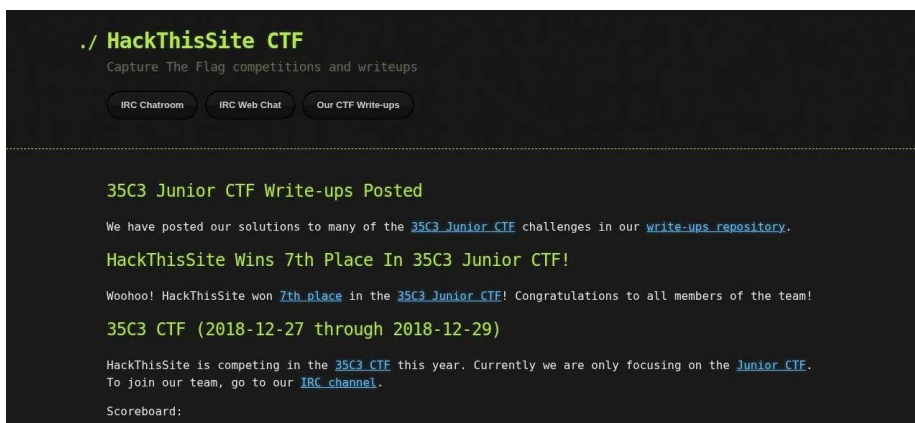


Figure 24: Captured screenshot — ctf.hackthissite.org

Description: GoWitness-captured screenshot of the CTF subdomain, a GitHub Pages-hosted Jekyll site publishing 35C3 Junior CTF write-ups.

Observation: This subdomain is hosted entirely on GitHub Pages/Fastly rather than the organization's own origin IP range, explaining why it returned different technology-detection results from the apex domain.

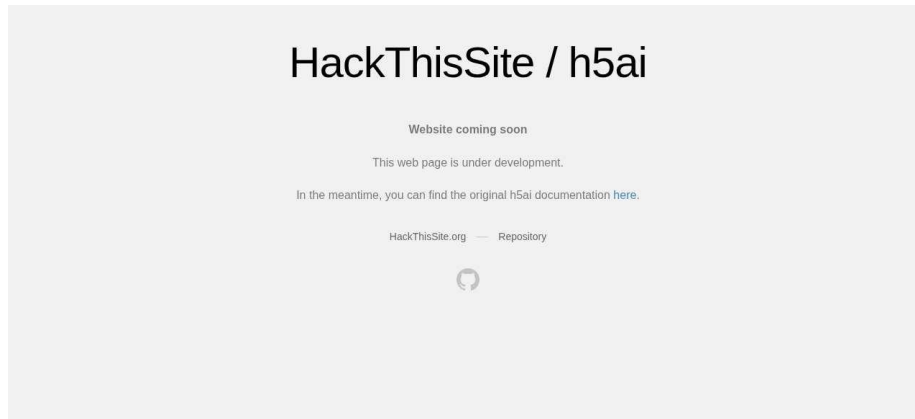


Figure 25: Captured screenshot — h5ai.hackthissite.org file mirror

Description: GoWitness-captured screenshot of the h5ai-powered file index/mirror subdomain, listing downloadable resources in a directory-style browser UI.

Observation: Unlike the intitle:"index of" dork in Task 6, this is a deliberately published file index (h5ai is a dedicated indexing application) rather than an accidental Apache directory listing.

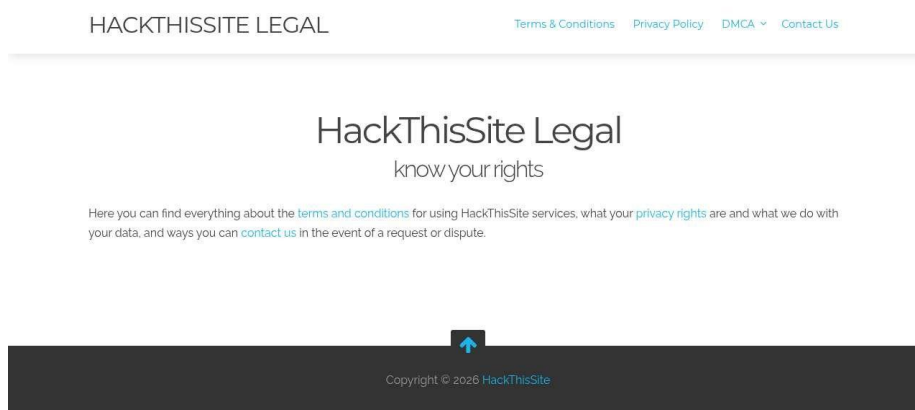


Figure 26: Captured screenshot — legal.hackthissite.org

Description: GoWitness-captured screenshot of the platform's Legal subdomain, presenting terms, conditions, and acceptable-use policy text.

Observation: This page is the authoritative source confirming the platform's terms for legitimate security testing and was cross-referenced before any task in this report was carried out.



Figure 27: Captured screenshot — forum.hackthissite.org

Description: GoWitness-captured screenshot of the community discussion forum subdomain, redirected automatically to HTTPS.

Observation: forum.hackthissite.org and forums.hackthissite.org are visually near-identical but returned different HTTP status codes (200 vs. 403) during probing, a distinction only the screenshots made easy to spot at a glance.

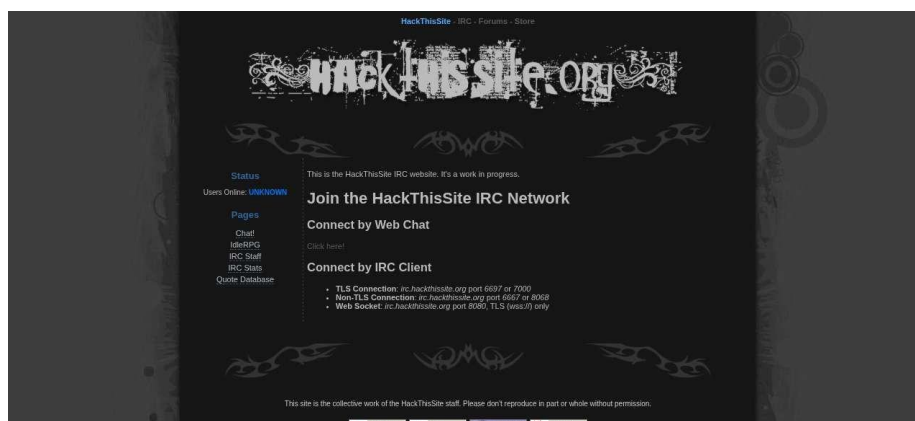


Figure 28: Captured screenshot — qdb.hackthissite.org

Description: GoWitness-captured screenshot of the IRC quotes-database (qdb) subdomain, sharing the same HackThisSite IRC landing page as several other irc-* hostnames.

Observation: Side-by-side review of this and the other irc-* screenshots was what first revealed that multiple distinct subdomains all serve an identical landing page, noted further below.

Observations

Reviewing the screenshot gallery side by side made it obvious within minutes that several differently-named subdomains (the various irc-*, *.irc.* and qdb names) actually serve the same IRC landing page — something that would have taken much longer to notice from httpx titles alone. This was the clearest case this week where the visual tool added information the text-based tools did not make obvious.

Key Learning Points

Visual recon scales human review in a way plain text output cannot; storing results in a queryable database (rather than just a folder of images) makes it possible to report on findings afterward instead of only looking at them once.

Security Relevance

A consolidated visual and metadata record like this is exactly what would be handed to a client or senior tester as engagement evidence, and it is also how a defender running their own external attack-surface review would catch forgotten or shadow web properties.

Technical Concepts Learned

CIA Triad

The foundational security model of Confidentiality, Integrity, and Availability. Every reconnaissance and testing activity ultimately exists to assess how well an organization protects these three properties; subdomain and credential exposure, for instance, are primarily confidentiality risks.

Footprinting

The broad process of collecting any publicly available information about a target organization — domain registration, employee details, technology choices — to build an overall profile before more targeted work begins. Tasks 4 and 6 of this week (WHOIS, Google dorking) are classic footprinting activities.

Reconnaissance

The first formal phase of the penetration testing lifecycle, covering both passive (no target interaction) and active (direct target interaction) information gathering. This entire week's task set sits inside this single phase.

Scanning

The follow-on phase where a tester actively probes discovered hosts to determine which services are running and on which ports. HTTP probing with httpx and httpprobe (Task 8) is the lightweight, web-focused edge of scanning; full port scanning was outside this week's scope.

Enumeration

The systematic extraction of detailed information from a discovered service — usernames, shares, DNS records, technology versions — once a target has been identified as live. WHOIS, DNS-record, and technology-fingerprinting tasks this week are all enumeration in this sense.

Vulnerability Assessment

The phase that takes enumerated technology and service details and matches them against known weaknesses (e.g., an outdated jQuery version, a leaked credential). This week's findings — old jQuery, CDN 400 errors — are exactly the kind of leads that phase would investigate further, even though performing that assessment itself was not part of Week 1.

Penetration Testing Lifecycle

The standard end-to-end structure of an engagement: scoping/authorization, reconnaissance, scanning, enumeration, vulnerability assessment, exploitation, post-exploitation, and reporting. This week corresponds entirely to the second stage, sitting on top of the explicit authorization already granted by HackThisSite's own published terms.

Risk Management

The discipline of weighing the likelihood and impact of a given exposure against the cost of fixing it. An outdated client-side library is lower risk than a leaked production credential, for example, which is why triage and prioritization matter more than simply listing every finding.

Security Principles

General defensive concepts such as least privilege, defense in depth, and minimizing attack surface, all of which this week's findings repeatedly connect back to — every extra forgotten subdomain or unmonitored CDN edge is, in effect, an avoidable increase in attack surface.

Challenges Faced

The table below documents the genuine issues encountered during the week's practical work, their root causes, how each was resolved, and the lesson taken from it.

Issue	Root Cause	Resolution	Lessons Learned
master.txt contained two non-domain lines ("FQDN:", "Session Scope").	amass_passive.txt held Amass's human-readable console banner text rather than a clean, names-only export.	Manually identified and excluded the two artifact lines when calculating the true candidate count (72 instead of 74).	Always inspect merged files before trusting their line count; use Amass's machine-readable output flags rather than its console banner for future merges.
Several CDN edge subdomains (*-v3dev-cdn, *-v3stage-cdn) returned HTTP 400 instead of a clean refusal.	These names point to a CDN/load-balancer edge that expects a specific Host-header routing rule not satisfied by a generic probe.	Documented them as "inconclusive / needs follow-up" rather than marking them dead, and cross-checked against httpx's tech-detect output.	Not every non-200 response means a dead host; CDN edges need a more targeted follow-up before being written off.
GitHub code searches for "password", "secret", and "api_key" returned thousands of loosely related results.	GitHub code search matches the domain and keyword independently across unrelated files, so most hits are incidental README mentions.	Manually triaged the top results from each query and confirmed none contained an actual HackThisSite credential before recording the search as "no leak found."	Plan extra review time for high-recall, low-precision OSINT sources; a large hit count is not itself a finding.
Subfinder, Assetfinder, Amass, and crt.sh each used a different output format (flat list vs. relationship graph vs. JSON).	No two reconnaissance tools format their output identically by default.	Standardized everything down to a flat, deduplicated text file before passing it to the next stage of the pipeline.	Build the normalization step into the workflow from day one rather than improvising it once formats start colliding.

Best Practices

A few security and workflow best practices stood out to me this week, several of them learned the hard way while debugging my own pipeline:

- Only perform reconnaissance, and any later testing, against targets you are explicitly authorized to test — such as HackThisSite's own published legal practice platform, or any system covered by a signed engagement scope.
- Prefer passive techniques before active ones wherever the engagement allows, to minimize the footprint left on target-side logs.
- Treat every automated tool's output as a hypothesis to verify, not a finished finding; cross-check at least one independent source before recording a result.
- Normalize and deduplicate data between pipeline stages so later tools always work against clean input.
- Apply the principle of least privilege to your own attack workstation and credentials, not only to the target's systems.
- Use long, unique, randomly generated passwords and a password manager for every tool or account involved in the engagement, including throwaway lab accounts.
- Keep a contemporaneous log of commands, timestamps, and output files as you go, rather than reconstructing it afterward — this report was only possible because every command and screenshot was captured during the work itself.
- Handle any findings, even from a legal practice site, with the same documentation discipline you would apply to a real client engagement, since the habits built in training carry over directly.

Weekly Summary Table

Task	Description	Status
1. Passive Subdomain Enumeration	Subfinder, Assetfinder, Amass (passive), crt.sh	Completed
2. Data Consolidation & Deduplication	Merged 4 sources into master.txt (72 valid candidate subdomains)	Completed
3. DNS Resolution, IP Mapping & Records	dig/nslookup/xargs — 52 unique IPs, MX/NS/TXT enumerated	Completed
4. WHOIS Enumeration	Registrar, lock status, creation/expiry dates, name servers	Completed
5. Technology Fingerprinting	WhatWeb + Wappalyzer cross-check	Completed
6. Search-Engine Reconnaissance	Google dorking — site:, filetype:, intitle:"index of"	Completed
7. GitHub Reconnaissance	Code/secret search across 3 keyword queries, manual triage	Completed
8. HTTP Probing & Live Host Discovery	httpx (35 hosts) + httpprobe (54 live URLs)	Completed
9. Visual Reconnaissance	GoWitness — 62 screenshots, 681 headers, 905 network logs	Completed

Conclusion

Week 1 of my Cyberster internship gave me a complete, working tour of the reconnaissance phase of a red-team engagement, built entirely around real tools applied to a real, authorized target rather than synthetic exercises. Starting from nothing more than a domain name, I was able to build a documented pipeline that produced a 72-entry candidate subdomain list, 52 resolved IP addresses, 54 confirmed live endpoints, a technology fingerprint, a WHOIS and DNS-record profile, search-engine and code-repository OSINT coverage, and a full visual screenshot archive with its own queryable database.

The most valuable lesson of the week was not any single tool, but the discipline of cross-checking and consolidating: no individual source (Subfinder, Amass, crt.sh, httpx, or GitHub search) was complete or fully trustworthy on its own, and the real skill was in merging, deduplicating, and triaging their combined output into something accurate enough to act on. I also learned that negative results — no open directory listing, no leaked secret found — are genuine, reportable findings rather than failed searches, and that small data-quality issues, like the artifact lines that slipped into master.txt, have to be caught by manually reviewing output rather than trusting line counts alone.

Technically, I am now comfortable running and chaining Subfinder, Assetfinder, Amass, crt.sh queries, dig/nslookup/whois, WhatWeb, httpx, httpprobe, and GoWitness inside a Kali Linux environment, and I have a clear mental model of how each tool's output feeds the next stage of an engagement. Going into Week 2, I feel ready to build on this reconnaissance foundation toward active scanning and the early stages of vulnerability assessment, with a recon workflow I can now reuse on any future authorized target.

References

- OWASP Foundation — OWASP Web Security Testing Guide (WSTG), Information Gathering chapter.
- NIST Special Publication 800-115 — Technical Guide to Information Security Testing and Assessment.
- MITRE ATT&CK — Reconnaissance (Tactic TA0043).
- ProjectDiscovery — Official Subfinder and httpx documentation.
- OWASP Amass Project — Official documentation and usage guide.
- Kali Linux Official Documentation.
- Sectigo / crt.sh — Certificate Transparency search service documentation.
- SensePost — GoWitness official documentation (v3).
- HackThisSite.org — Terms & Conditions and Project Guide.
- ICANN — WHOIS Status Codes reference (icann.org/epp).

Appendix

A. Raw Evidence Files Retained

The following text and binary evidence files generated during Week 1 are retained alongside this report and were the direct source for every figure and statistic cited above: subfinder.txt, assetfinder.txt, amass_passive.txt, amass_active.txt, crtsh.txt, master.txt, ips.txt, resolvers.txt, live.txt, httpx_full.txt, and gowitness.sqlite3.

B. Full Screenshot Archive

GoWitness captured 62 site screenshots in total, stored in the project's screenshots/ directory. Figures 23–28 in the Task 9 section present a representative sample (the homepage, the CTF write-up subdomain, the h5ai file mirror, the Legal subdomain, the community forum, and the IRC quotes-database page); the remaining images cover every other live host confirmed in Task 8 and are available in the same directory for reference.

C. Terminal & Browser Screenshot Index

Figures 1–22 in the body of this report correspond one-to-one with the dated screenshot files captured during the week's work (named Screenshot_2026-06-24_HH_MM_SS.png in the project archive), in the same order they were taken.

D. Submission Checklist

- Cover Page — included
- Table of Contents — included (auto-generated field; update in Word before submitting)
- Executive Summary — included
- Learning Objectives — included
- Environment Setup — included, with tools table
- Task Documentation (9 tasks) — included, with objective/theory/tools/procedure/commands/output/screenshots/observations/key learnings/security relevance for each
- Commands — included in shaded code blocks for every task
- Screenshots with Captions — included (28 figures with description and observation)
- Weekly Summary Table — included
- References — included
- Grammar Check — recommended before final submission
- Export as PDF — recommended alongside the .docx for final submission, per the Cyberster naming convention Week01_[YourName]_Report.pdf